CAMPUSKEY: Smart RFID Attendance & Identity System

Master Team Blueprint, System Architecture, Hardware Pinout & Complete Reference Code
Institution: ICFAI School of Science and Technology | **Program:** BCA | **Course:** ICSD (Introduction to Computer System Design)

1. Project Overview & Core Architecture

CAMPUSKEY is an integrated hardware-software IoT attendance and access management prototype built for both **Students** and **Faculty** at the ICFAI School of Science and Technology. Designed for the **Introduction to Computer System Design (ICSD)** curriculum, the system demonstrates how an embedded microcontroller unit communicates across three distinct hardware buses—**SPI**, **I2C**, and **Digital GPIO**—while synchronizing state in real time with a cloud PostgreSQL database and an administrative web portal.

System Layer	Core Technology	Architectural Responsibility in CAMPUSKEY
Embedded MCU	ESP8266 (NodeMCU v3, 80 MHz)	Polls RC522 via SPI, drives 16×2 LCD via I2C, controls LEDs/Buzzer via GPIO, executes HTTPS REST/RPC requests, and caches offline errors in flash memory.
RFID Sensing Layer	MFRC522 (13.56 MHz) + MIFARE 1K	Reads the 4-byte hardware Unique ID (UID) and verifies Sector 1 authentication blocks when a student or faculty member taps their ID card.
Local User Feedback	16×2 I2C LCD + 5V Buzzer + 2 LEDs	Displays 16-character status/error messages, countdown timers, and distinct audio-visual pass/fail cues immediately at the scanning terminal.
Backend & Database	Supabase (PostgreSQL + RPC + Realtime)	Stores Student/Faculty profiles, daily attendance logs, failed scan diagnostic logs, and the live `system_state` flag (Attendance vs. Registration mode).
Security & Auth	Supabase Auth + Row Level Security	Protects admin routes via JWT login and restricts ESP8266 database access to a single `SECURITY DEFINER` RPC function authenticated by a device key.
Admin Web Portal	HTML5, CSS3, Vanilla JS on Vercel	Provides 4 dedicated pages (Overall Dashboard, Attendance Page with CSV/Excel export, Users & 60s Registration Page, and Error Diagnostics Page).

2. Hardware Components, Electrical Rules & Pinout Mapping

The prototype uses the nine hardware components already acquired by the team. To guarantee electrical safety and avoid the ESP8266 GPIO pin shortage (the NodeMCU has 9 safe digital GPIOs while all modules nominally require 10), our design incorporates three essential hardware engineering rules:

- **Rule 1 — Tie RC522 RST to 3.3V to Save One GPIO Pin:** Connecting the RC522 `RST` pin directly to the breadboard `3.3V` rail frees `D3` (GPIO 0) so the NodeMCU can independently drive the Red LED, Green LED, and 5V Buzzer without running out of pins.
- **Rule 2 — Strictly Separate 3.3V and 5V Power Rails:** The **RC522 RFID module MUST be powered from `3V3` (3.3V)**—connecting it to 5V will burn out the chip. Conversely, the **16×2 I2C LCD and 5V Active Buzzer MUST be powered from `VIN` / `VU` (5V USB power)** so the LCD backlight and contrast remain crisp.
- **Rule 3 — Expand Jumper Wires from 14 to 20 Wires:** The I2C LCD uses all 4 Male-to-Female jumper wires. Connecting the RC522 (7 pins), buzzer, 2 LEDs with 220 Ω resistors, and power rails requires **16 Male-to-Male jumper wires** (add 6 extra Male-to-Male wires to the 10 currently in the kit).

Hardware Component	Component Pin	NodeMCU Pin	ESP8266 GPIO	Bus Protocol & Wiring Notes
RC522 RFID Reader	`3.3V (VCC)`	`3V3`	Power Rail	Strictly 3.3V DC (Never wire to `VIN` 5V)
RC522 RFID Reader	`RST`	`3V3`	Power Rail	Tied high to 3.3V rail (saves 1 GPIO pin; soft-reset via SPI)
RC522 RFID Reader	`GND`	`GND`	Ground Rail	Connected to common breadboard ground rail
RC522 RFID Reader	`MISO`	`D6`	GPIO 12	Hardware SPI Master-In Slave-Out
RC522 RFID Reader	`MOSI`	`D7`	GPIO 13	Hardware SPI Master-Out Slave-In
RC522 RFID Reader	`SCK`	`D5`	GPIO 14	Hardware SPI Serial Clock (13.56 MHz carrier)
RC522 RFID Reader	`SDA (SS/NSS)`	`D8`	GPIO 15	Hardware SPI Chip Select (must stay low at boot)
16×2 I2C LCD Display	`VCC`	`VIN (VU)`	5V USB Rail	5V required for readable LCD contrast (use M-F wire)
16×2 I2C LCD Display	`GND`	`GND`	Ground Rail	Common ground rail (use M-F wire)
16×2 I2C LCD Display	`SCL`	`D1`	GPIO 5	Hardware I2C Clock line (use M-F wire)

Hardware Component	Component Pin	NodeMCU Pin	ESP8266 GPIO	Bus Protocol & Wiring Notes
16×2 I2C LCD Display	SDA	D2	GPIO 4	Hardware I2C Data line (use M-F wire, Addr 0x27)
Green Status LED	Anode (+)	D0	GPIO 16	In series with 220 Ω resistor to GND (Access Granted)
Red Status LED	Anode (+)	D3	GPIO 0	In series with 220 Ω resistor to GND (Error / Denied)
5V Active Buzzer	Positive (+)	D4	GPIO 2	Digital HIGH trigger; negative pin to GND

3. Complete System Working: Dual-Mode Operational Logic

CAMPUSKEY divides its operation into two distinct modes controlled by a centralized `system_state` table in Supabase. Whatever fails during either mode, the system immediately halts the transaction, displays the exact reason on the 16×2 LCD, triggers a distinct LED/Buzzer alert, logs the fault for the Admin Error Page, and guarantees no corrupt or unauthorized attendance is saved.

Part 1: Normal Attendance Mode (When an Existing User Scans a Card)

1. **Card Tap & RF Coupling:** The student or faculty member taps their RFID card against the RC522 antenna. If the card is pulled away too fast or fails the 16-bit CRC checksum, the ESP8266 stops immediately, displays `CARD READ ERROR` / `Hold Card Steady` on the LCD, blinks the Red LED with 3 fast beeps, and logs a `CARD_READ_FAIL` event.
2. **API Transmission to Supabase:** Once the 4-byte hexadecimal UID (e.g., `A34F9B12`) is read, the ESP8266 sends an HTTPS POST request to the Supabase RPC function `handle_card_scan`. If Wi-Fi is disconnected or the request fails to reach the backend, the ESP8266 shows `API / WIFI ERROR` / `Check Network` on the LCD, lights the Red LED, stores the failed event in internal `LittleFS` flash memory so it can be uploaded to the Error Page once Wi-Fi reconnects, and aborts without marking attendance.
3. **User Existence Verification:** The backend queries the `users` table for `card_uid`. If no matching student or faculty record exists, the backend inserts an `UNREGISTERED_UID` entry into `error_logs` and returns an error. The LCD displays `USER NOT FOUND` / `Unregistered ID`, lights the Red LED, sounds 1 long beep (800 ms), and records zero attendance.
4. **Duplicate Scan Prevention:** If the user exists, the backend checks `attendance_logs` for `user_id` on `CURRENT_DATE`. If the user has already scanned today, the backend logs a `DUPLICATE_SCAN` warning in `error_logs` and returns `ALREADY_RECORDED`. The LCD shows `ALREADY RECORDED` / `Done For Today` with 2 medium Red LED beeps.
5. **Attendance Confirmation:** When all checks pass, Supabase inserts a verified attendance row with timestamp and role, returning the user's name. The ESP8266 turns ON the Green LED for 2 seconds, sounds 1 crisp 150 ms beep, and displays `HEY, [FIRST_NAME]` on LCD Line 1 and `Present Marked!` on Line 2.

Part 2: New User Registration Mode (60-Second Admin Window)

Important RFID Technical Reality: Brand-new MIFARE Classic 1K cards from an RC522 kit only contain a factory-burned 4-byte UID—their data sectors are blank until programmed. Therefore, CAMPUSKEY combines automatic hardware UID extraction with instant web dashboard profile editing:

1. **Admin Clicks "New Registration":** When the admin clicks the **New Registration** button on the Users Page, the website sets `system_state.mode = 'REGISTRATION'` and `expires_at = NOW() + 60 seconds` in Supabase, opening a live 60-second registration modal on the dashboard.
2. **1-Minute Countdown & Auto-Timeout:** The ESP8266 detects Registration Mode during its status poll, turns ON both Green and Red LEDs, and counts down `NEW REGISTRATION` / `Scan in: XXs` on the LCD. If nobody scans a card within 60 seconds, the timer expires, the LCD shows `REG. TIMED OUT` / `Back to Normal`, and both the hardware and website automatically return to Normal Attendance Mode.
3. **Card Scan & Conflict Check:** When the new user taps their card within the 1-minute window:
   - **If Card Read Fails or API Drops:** Shows `CARD READ ERROR` or `API / WIFI ERROR` on the LCD, stops registration, and reverts to Normal Attendance Mode.
   - **If User/Card Already Exists:** If the scanned UID is already assigned to another student or faculty member, Supabase logs a `REG_CONFLICT` error, the LCD displays `USER EXISTS!` / `Card In Use` with a long Red beep, and the system reverts to Normal Attendance Mode.
   - **If Card is New (Success):** Supabase generates a unique `UUID`, registers the new user linked to that `card_uid`, switches `system_state` back to `ATTENDANCE`, and pushes the captured UID directly into the Admin Dashboard form via Supabase Realtime so the admin can fill in or edit Student details (Name, Branch, Course, Enrollment No., Mobile) or Faculty details (Name, Department/Branch, Role, Mobile). The LCD shows `CARD REGISTERED` / `UID: [HEX_UID]` with 2 quick Green beeps.

4. Complete Hardware Feedback & Error Diagnostics Matrix

Because a 16×2 LCD strictly fits **16 characters per line**, all messages are engineered to fit cleanly without scrolling or truncation while pairing with LED and Buzzer patterns and automatic hardware wire checks:

System Event / Fault	LCD Line 1 (16ch)	LCD Line 2 (16ch)	LED Status	Buzzer Cue	System Action
Normal Idle State	`CAMPUSKEY READY`	`Tap ID Card...`	Both OFF	Silent	Polls RC522 & state
New Reg. Mode (60s)	`NEW REGISTRATION`	`Scan in: 59s`	Green + Red ON	1 short chirp	60s countdown active
Valid Attendance	`HEY, RAHUL`	`Present Marked!`	Green ON (2s)	1 short beep	Saves attendance row
New User Registered	`CARD REGISTERED`	`UID: A34F9B12`	Green ON (2s)	2 quick beeps	Creates UUID + profile

System Event / Fault	LCD Line 1 (16ch)	LCD Line 2 (16ch)	LED Status	Buzzer Cue	System Action
Already Scanned	`ALREADY RECORDED`	`Done For Today`	Red ON (2s)	2 medium beeps	Logs `DUPLICATE_SCAN`
Unregistered Card	`USER NOT FOUND`	`Unregistered ID`	Red ON (2s)	1 long beep	Logs `UNREGISTERED_UID`
Card Exists (Reg.)	`USER EXISTS!`	`Card In Use`	Red ON (2s)	1 long beep	Exits Reg. → Normal
Reg. 60s Timeout	`REG. TIMED OUT`	`Back to Normal`	Red Blink (1s)	1 low beep	Exits Reg. → Normal
Card Read Failed	`CARD READ ERROR`	`Hold Card Steady`	Red 3x Blink	3 fast beeps	Logs `CARD_READ_FAIL`
API / Wi-Fi Offline	`API / WIFI ERROR`	`Check Network`	Red Solid ON	2 long beeps	Caches to `LittleFS`
RC522 Wire Loose	`ERR: RFID WIRE`	`Check SPI Pins`	Red Solid ON	Continuous pulse	SPI `0x37` self-test fail
LCD I2C Wire Loose	*(LCD Unpowered)*	*(LCD Unpowered)*	Red 3x Repeat	3x Repeat Beeps	I2C bus ACK failure

5. Software Website Architecture & Security Upgrades

Instead of coupling the ESP8266 directly to Google Sheets (which suffers from slow redirect latency and fragile script quotas), CAMPUSKEY stores all records in **Supabase PostgreSQL** and provides a **1-Click Export to CSV / Google Sheets / Excel** button directly inside the Admin Dashboard.

The 4 Core Pages of the Admin Web Application

- **Page 1 — Overall Admin Dashboard (`index.html`):** Protected by **Supabase Auth** login. Displays 4 live metric counters (Total Registered Students, Total Registered Faculty, Today's Attendance Count & Percentage, Today's Failed Scan Count), a Monthly Attendance Bar/Line Chart by branch, and a live real-time scan activity stream.
- **Page 2 — Attendance Management Page (`attendance.html`):** Features dedicated tabs for **Students** and **Faculty**, a month/date-range selector, automatic calculation of each person's monthly attendance percentage (highlighting students below 75%), manual attendance adjustment by the admin, and instant CSV/Excel spreadsheet export.
- **Page 3 — Users & Registration Page (`users.html`):** Separate Student and Faculty directories where every user has a unique system `UUID` alongside their `card_uid`. Student records store *Full Name, Branch, Course (BCA), Enrollment Number, and Mobile Number*; Faculty records store *Full Name, Department/Branch, Role/Designation, Employee Roll/ID, and Mobile Number*. Includes full Edit/Delete controls plus the **New Registration (60s Scan)** button.

- **Page 4 — Error & Hardware Diagnostics Page (`errors.html`):** Displays every failed scan logged in `error_logs` with Timestamp, Card UID, Matched User (if any), Failure Type (`DUPLICATE_SCAN`, `UNREGISTERED_UID`, `REG_CONFLICT`, `CARD_READ_FAIL`, `WIFI_OFFLINE_SYNC`), and recommended hardware/software remediation steps.

4 Key Security Enhancements (For Prototype & Viva Defense)

- **Supabase Row Level Security (RLS) + Device Secret RPC:** Public frontend code only holds the Supabase `anon` key with RLS policies restricting read/write access to authenticated admins. The ESP8266 never touches raw tables; it calls a locked PostgreSQL RPC function (`handle_card_scan`) requiring a secret `p_device_key`.
- **MIFARE Sector 1 Anti-Cloning Verification:** Standard 4-byte UIDs can be copied onto cheap magic cards. During registration, the ESP8266 can write a 16-byte institutional signature (`CAMPUSKEY_ICFAI1`) into **Sector 1, Block 4** protected by a custom 6-byte Crypto1 key, preventing cloned UIDs from passing verification.
- **Offline Flash Queue (`LittleFS`) for Network Dropouts:** When Wi-Fi drops, failed scan attempts are written to `/offline_errors.txt` in ESP8266 flash memory and automatically uploaded to the Error Page once Wi-Fi reconnects.
- **Continuous Hardware Watchdog (Power-On Self-Test):** Every 5 seconds, the MCU checks the I2C bus ACK from the LCD (`0x27`) and reads the RC522 silicon version register (`0x37`). Any loose breadboard wire triggers immediate visual/audio diagnostics.

---

6. Complete Project Reference Code

Below is the complete, production-ready code for all three layers of **CAMPUSKEY**: (A) the Supabase SQL Database Schema & RPC Engine, (B) the ESP8266 NodeMCU Arduino C++ Firmware, and (C) the Frontend Admin Dashboard JavaScript Controller.

A. Supabase PostgreSQL Schema & RPC Backend (`supabase_setup.sql`)

Run this script in the **Supabase SQL Editor** to create all tables, enable Row Level Security, and deploy the atomic dual-mode scan engine:

```
-- 1. Enable UUID extension
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

-- 2. Users Table (Stores both Students and Faculty with unique UUID)
CREATE TABLE public.users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    card_uid VARCHAR(32) UNIQUE NOT NULL,
    user_type VARCHAR(16) NOT NULL CHECK (user_type IN ('STUDENT', 'FACULTY')),
    full_name VARCHAR(120) NOT NULL DEFAULT 'Pending Admin Edit',
    branch VARCHAR(80) NOT NULL DEFAULT 'BCA / Science & Tech',
    course_or_role VARCHAR(80) NOT NULL DEFAULT 'BCA',
    enrollment_or_roll VARCHAR(50) UNIQUE,
    mobile_number VARCHAR(20),
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- 3. Attendance Logs Table (One verified scan per user per day)
```